ADVANCED JAVA · SPRING BOOT MINI PROJECT

Student Placement Management System

A web portal that replaces the registers of a college placement cell - student records, visiting recruiters, campus drives, applications and the final placement report.

Java 17 · Spring Boot 3.3
Spring MVC, Spring Data JPA

Thymeleaf + CSS
Server rendered multi page UI

5 tables in 3NF
H2 in memory, MySQL ready

Waterfall model
Six phases, one deliverable each

CONTENTS

What this presentation covers

1. **Introduction** - problem, objective, scope and users
2. **Technology and architecture** - layers and request flow
3. **Waterfall model** - how the project was built
4. **Coding** - entity, repository, service, controller, view
5. **Screens** - dashboard, CRUD pages, reports
6. **Database description** - ER design, normalisation, joins
7. **Testing, results and future scope**

PROJECT AT A GLANCE

Modules	Students, Companies, Drives, Applications, Reports
Java classes	26 (entity, repository, service, controller, DTO, exception)
Pages	16 Thymeleaf templates, 12 screens
Tables	department, company, student, drive, application
CRUD	Full create / read / update / delete on 4 entities
Tests	5 JUnit tests, all passing

Problem, objective and scope

THE PROBLEM TODAY

- Student details, company visits and results are kept in separate Excel sheets and registers.
- Eligibility for a drive (CGPA and backlog cut off) is checked by hand for every student.
- The same student sometimes gets entered twice for one company.
- Branch wise placement percentage has to be counted manually at the end of the season.

OBJECTIVE

- One portal where the placement officer manages students, recruiters and drives.
- Eligibility and duplicate checks done by the system, not by the officer.
- Every application tracked through Applied → Shortlisted → Selected / Rejected.
- Reports generated the moment the page is opened.

SCOPE OF THIS BUILD

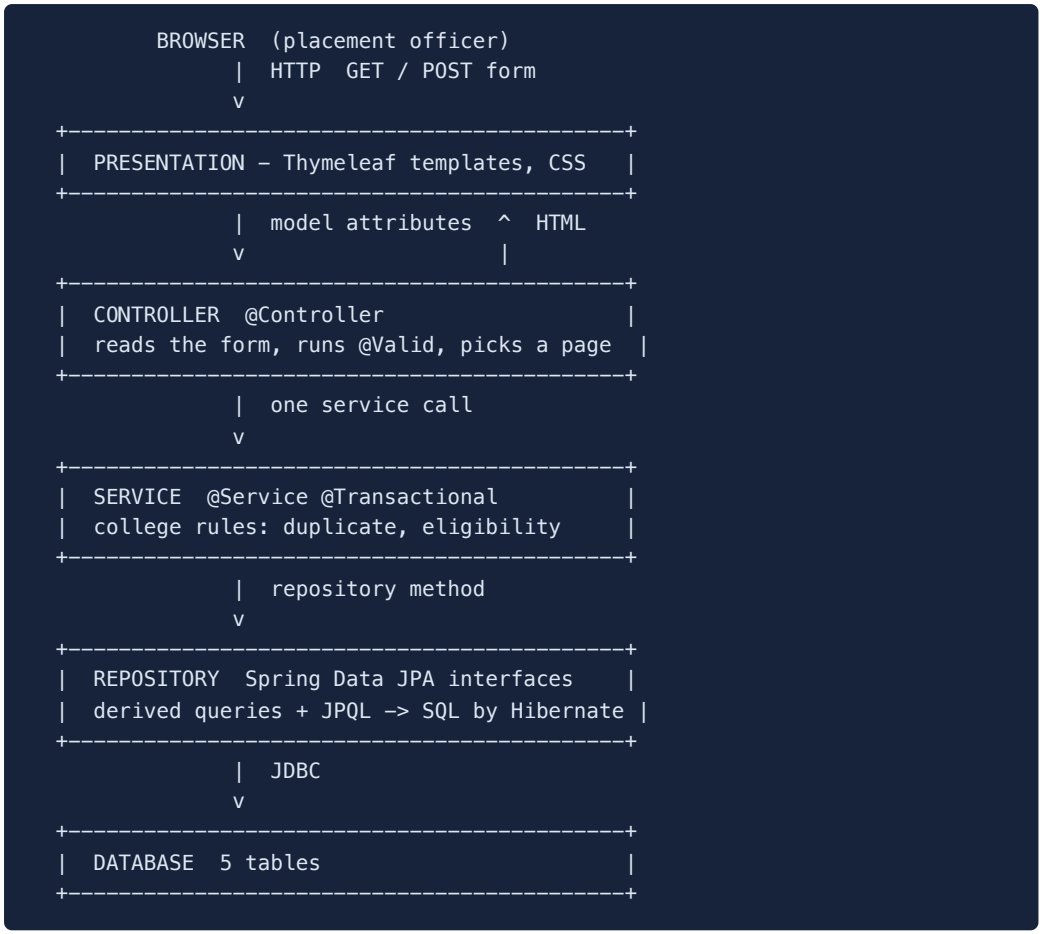
- CRUD screens for students, companies, drives and applications.
- Search and filter on the list pages, drive status life cycle.
- Automatic eligible student list for every drive.
- Dashboard and report page with placement statistics.

USERS OF THE SYSTEM

ROLE	RESPONSIBILITY
Placement Officer	Master data - students, recruiters, drives; opens and closes a drive
Placement Coordinator	Enters applications, updates the interview stage and remarks
Student (future scope)	Read only view of own profile and eligible drives

Out of scope - login and role based access, resume file upload and email alerts. These are listed as future scope on the last slide.

Layered architecture



RULES KEPT WHILE CODING

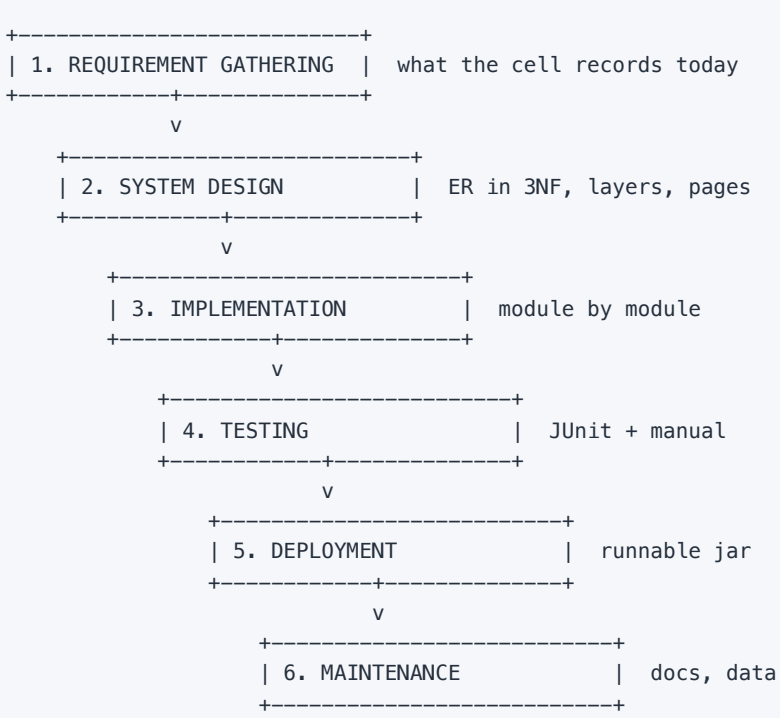
- A controller never calls a repository directly.
- A service never reads the request and never returns HTML.
- Only the entity classes know table and column names.

FLOW OF ONE REAL ACTION - SAVING A STUDENT

```
POST /students/save
 1  StringToDepartmentConverter: "1" -> Department
 2  @Valid checks the Student annotations
    errors -> back to students/form with messages
 3  StudentService.create()
      checkRollNumberIsFree()
      checkEmailIsFree()
      studentRepository.save()
 4  Hibernate: insert into student (...) values (...)
 5  flash message + redirect to /students/{id}
    (redirect after POST, so refresh cannot
     insert the same student twice)
```

3. SOFTWARE PROCESS

Waterfall model followed



Why waterfall suits this project - the working of a placement cell is fixed and fully known before coding, so each phase could be completed and signed off before the next one started.

PHASE	DELIVERABLE
Requirement	Module list, actors, 12 screens, report list
Design	5 table ER design in 3NF, layered packages, URL plan
Implementation	26 Java classes, 16 templates - student, company, drive, application, reports
Testing	5 JUnit tests + manual test of every CRUD path
Deployment	`mvn clean package` → `placement-portal.jar` with embedded Tomcat
Maintenance	Sample data, documentation, screenshots, future scope

TEAM RESPONSIBILITY

MEMBER	OWNED WORK
1	Database design, entities, repositories
2	Service layer rules, exceptions, JUnit tests
3	Controllers, form binding, validation
4	Thymeleaf pages, CSS, documentation

Entity and repository layer

ENTITY - STUDENT MAPPED TO A TABLE

```
@Entity
@Table(name = "student", uniqueConstraints = {
    @UniqueConstraint(name = "uk_student_roll",
                      columnNames = "roll_number") })
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Roll number is required")
    @Column(name = "roll_number", nullable = false)
    private String rollNumber;

    @NotNull @DecimalMax(value = "10.0",
        message = "CGPA cannot be more than 10")
    private Double cgpa;

    // many students belong to one branch
    @ManyToOne(optional = false)
    @JoinColumn(name = "department_id", nullable = false)
    private Department department;

    @OneToMany(mappedBy = "student",
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Application> applications = new ArrayList<>();
```

REPOSITORY - NO SQL WRITTEN BY HAND

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    // method name itself becomes the where clause
    boolean existsByRollNumberIgnoreCase(String roll);
    List<Student> findAllByOrderByRollNumberAsc();

    // students who clear the cut off of a drive
    // and have not applied to it yet
    @Query("""
        select s from Student s
        where s.cgpa >= :minCgpa
          and s.backlogs <= :maxBacklogs
          and s.id not in (select a.student.id
                           from Application a
                           where a.drive.id = :driveId)
        order by s.cgpa desc
        """)
    List<Student> findEligibleStudents(
            @Param("minCgpa") Double minCgpa,
            @Param("maxBacklogs") Integer maxBacklogs,
            @Param("driveId") Long driveId);
}
```

Why JPA and not plain JDBC - no connection handling, no result set to map by hand, and the same code runs on H2 during the demo and on MySQL in the lab.

Service rules, controller and view

SERVICE - THE COLLEGE RULES LIVE HERE

```
@Service
@Transactional(readOnly = true)
public class ApplicationService {

    @Transactional
    public Application create(Long studentId, Long driveId,
                              LocalDate appliedOn, String remarks) {
        Student student = studentService.getById(studentId);
        Drive drive = driveService.getById(driveId);

        checkNotAlreadyApplied(studentId, driveId);
        checkEligibility(student, drive);
        ...
    }

    private void checkEligibility(Student s, Drive d) {
        if (!d.isAcceptingApplications())
            throw new EligibilityNotMetException(...);
        if (s.getCgpa() < d.getMinCgpa())
            throw new EligibilityNotMetException(...);
        if (s.getBacklogs() > d.getMaxBacklogs())
            throw new EligibilityNotMetException(...);
    }
}
```

CONTROLLER - REQUEST HANDLING ONLY

```
@PostMapping("/save")
public String saveStudent(
        @Valid @ModelAttribute("student") Student student,
        BindingResult bindingResult, Model model,
        RedirectAttributes redirectAttributes) {

    boolean isNew = student.getId() == null;
    if (bindingResult.hasErrors()) return "students/form";

    try {
        Student saved = isNew
            ? studentService.create(student)
            : studentService.update(student.getId(), student);
        redirectAttributes.addFlashAttribute("successMessage", ...);
        return "redirect:/students/" + saved.getId();
    } catch (BusinessRuleException e) {
        model.addAttribute("errorMessage", e.getMessage());
        return "students/form";
    }
}
```

VIEW - THYMELEAF BINDS TO THE SAME OBJECT

```
<form method="post" th:action="@{/students/save}"
      th:object="${student}">
  <input type="text" th:field="*{rollNumber}">
  <span class="field-error"
        th:if="${#fields.hasErrors('rollNumber')}"
        th:errors="*{rollNumber}"></span>
</form>
```

Coding practices and validation

CONVENTIONS USED

ITEM	CONVENTION	EXAMPLE
Class	PascalCase, noun	`ApplicationService`
Method	camelCase, verb	`findEligibleStudents()`
Constant	UPPER_SNAKE	`DISPLAY_DATE`
Table / column	snake_case	`roll_number`
Package	lower case, by layer	`com.college.placement.service`
URL	same pattern per module	`/students/{id}/edit`

PRACTICES

- Constructor injection, no field `@Autowired`.
- `@Transactional(readOnly = true)` on the class, write methods override it.
- Update copies form fields on to the loaded row, so child rows are never wiped.
- Redirect after POST with flash messages.
- Comments explain the reason, not the syntax.

TWO LEVELS OF CHECKING

Field level - Bean Validation on the entity, run by `@Valid`: compulsory fields, valid email, phone exactly 10 digits, CGPA 0 to 10, batch year 2015 to 2035, backlogs not negative.

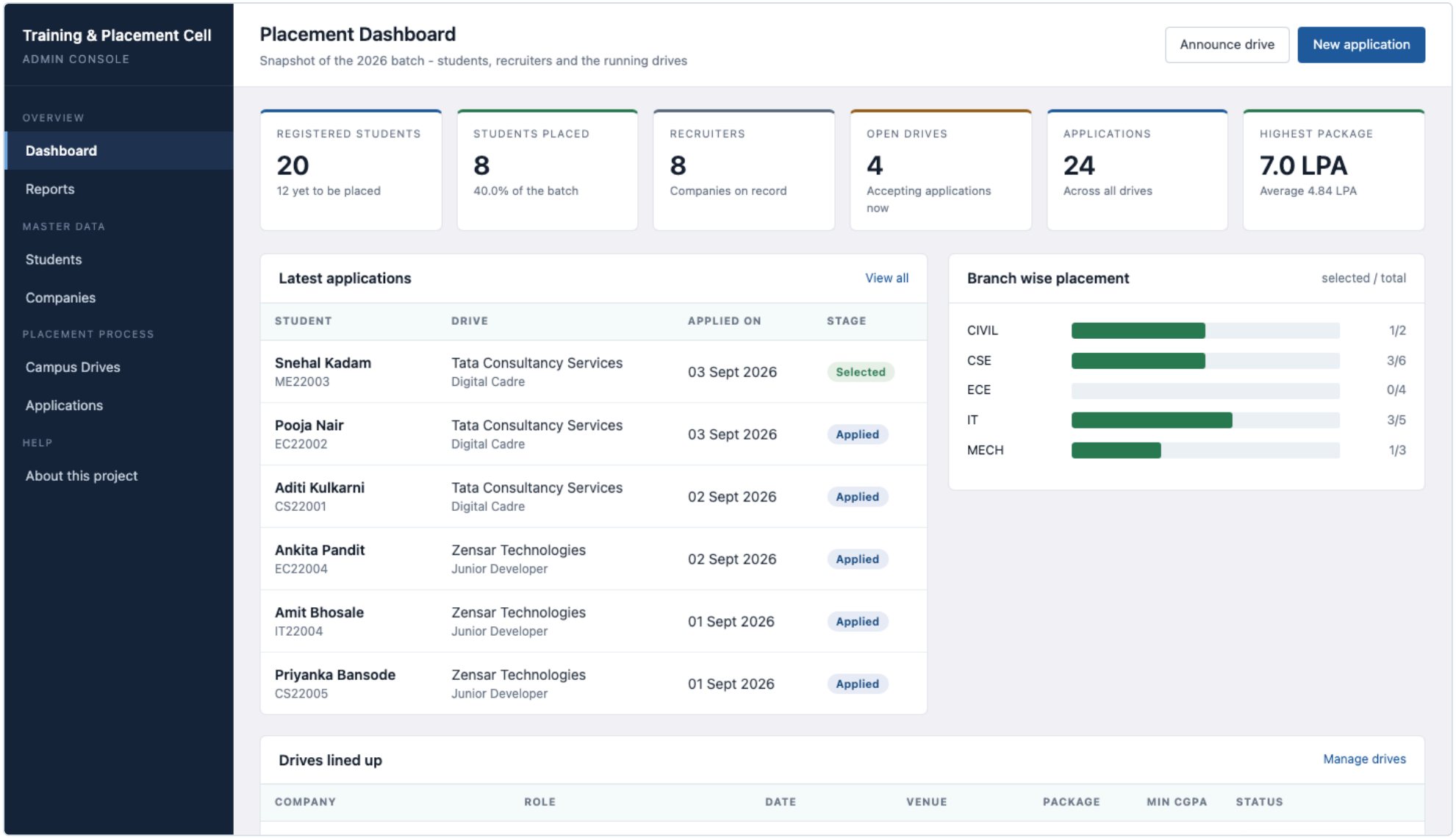
Rule level - in the service, so the rule holds even if the form is bypassed:

1. Roll number and email unique
2. Company name unique
3. Drive must be **Open** to accept applications
4. CGPA ≥ drive cut off
5. Backlogs ≤ allowed backlogs
6. No student twice in one drive (also a UNIQUE key)

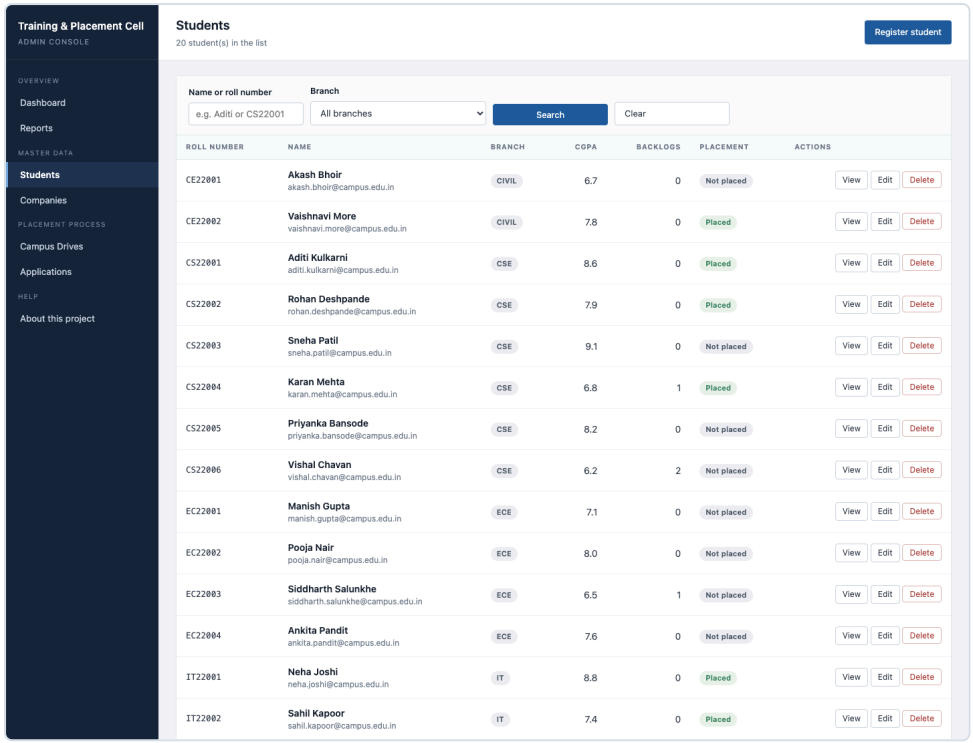
ERROR HANDLING

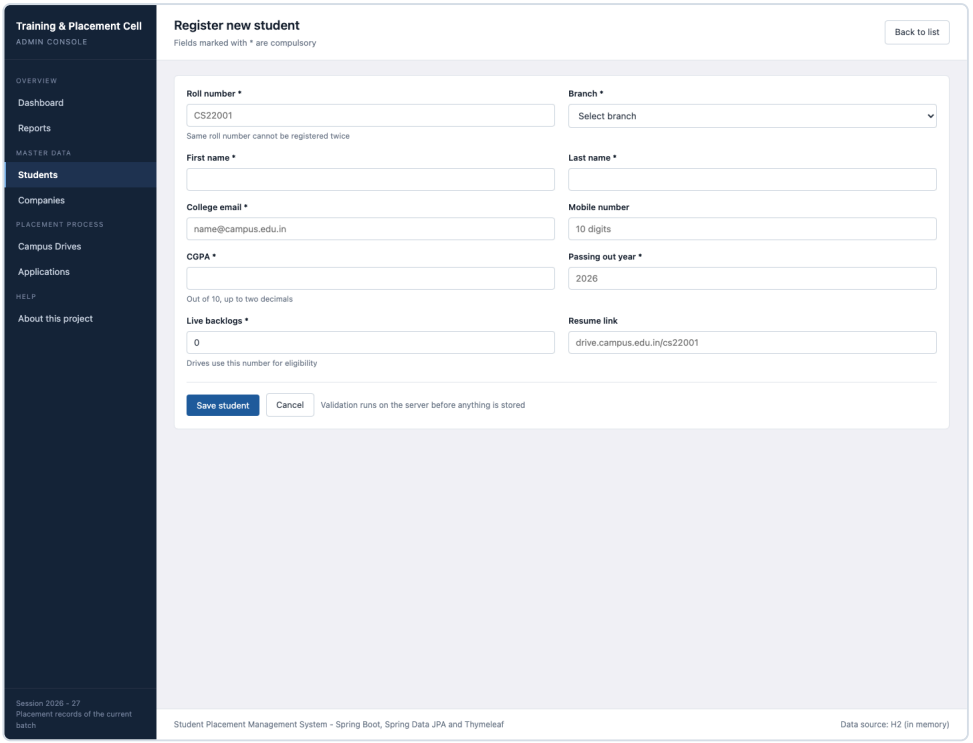
`@ControllerAdvice` catches `ResourceNotFoundException` (404), `BusinessRuleException` (409) and anything unexpected (500), and shows the project error page instead of a stack trace.

Dashboard

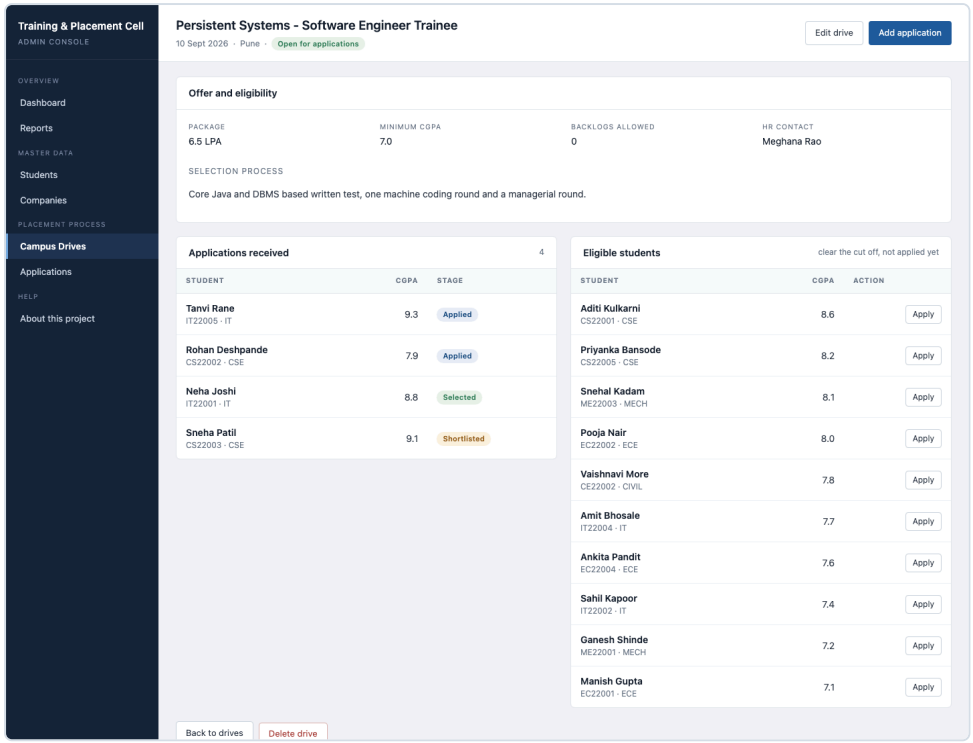

Training & Placement Cell
ADMIN CONSOLE
OVERVIEW
Dashboard
Reports
MASTER DATA
Students
Companies
PLACEMENT PROCESS
Campus Drives
Applications
HELP
About this project
Placement Dashboard
Snapshot of the 2026 batch - students, recruiters and the running drives
Announce drive
New application
REGISTERED STUDENTS
20
12 yet to be placed
STUDENTS PLACED
8
40.0% of the batch
RECRUITERS
8
Companies on record
OPEN DRIVES
4
Accepting applications now
APPLICATIONS
24
Across all drives
HIGHEST PACKAGE
7.0 LPA
Average 4.84 LPA
Latest applications
View all
STUDENT
DRIVE
APPLIED ON
STAGE
Snehal Kadam
ME22003
Tata Consultancy Services
Digital Cadre
03 Sept 2026
Selected
Pooja Nair
EC22002
Tata Consultancy Services
Digital Cadre
03 Sept 2026
Applied
Aditi Kulkarni
CS22001
Tata Consultancy Services
Digital Cadre
02 Sept 2026
Applied
Ankita Pandit
EC22004
Zensar Technologies
Junior Developer
02 Sept 2026
Applied
Amit Bhosale
IT22004
Zensar Technologies
Junior Developer
01 Sept 2026
Applied
Priyanka Bansode
CS22005
Zensar Technologies
Junior Developer
01 Sept 2026
Applied
Branch wise placement
selected / total
CIVIL 1/2
CSE 3/6
ECE 0/4
IT 3/5
MECH 1/3
Drives lined up
Manage drives
COMPANY
ROLE
DATE
VENUE
PACKAGE
MIN CGPA
STATUS

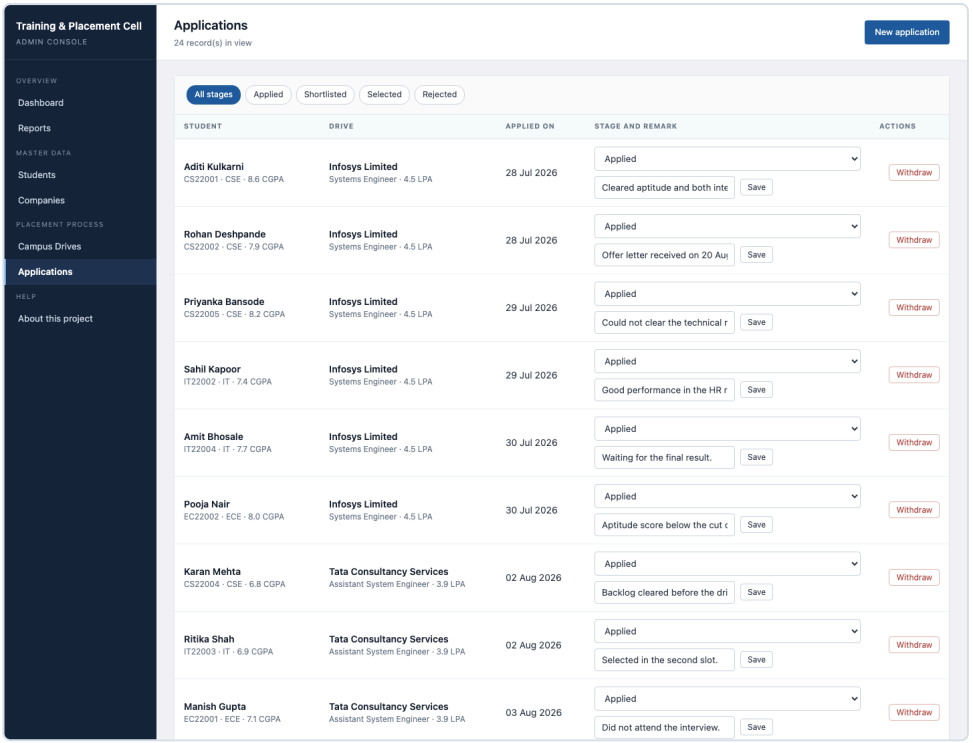
Student list and student form



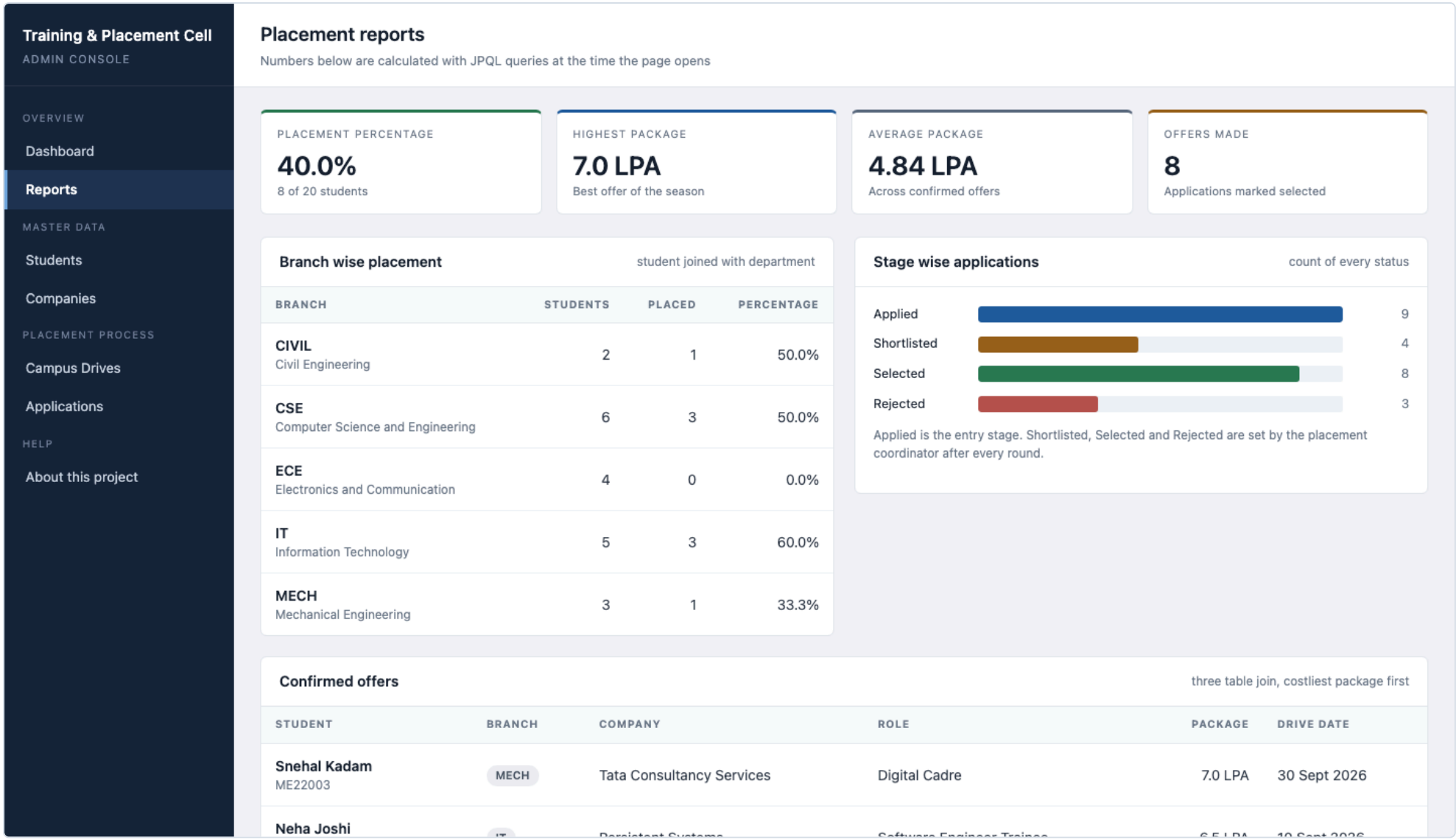


Drive details and application tracking

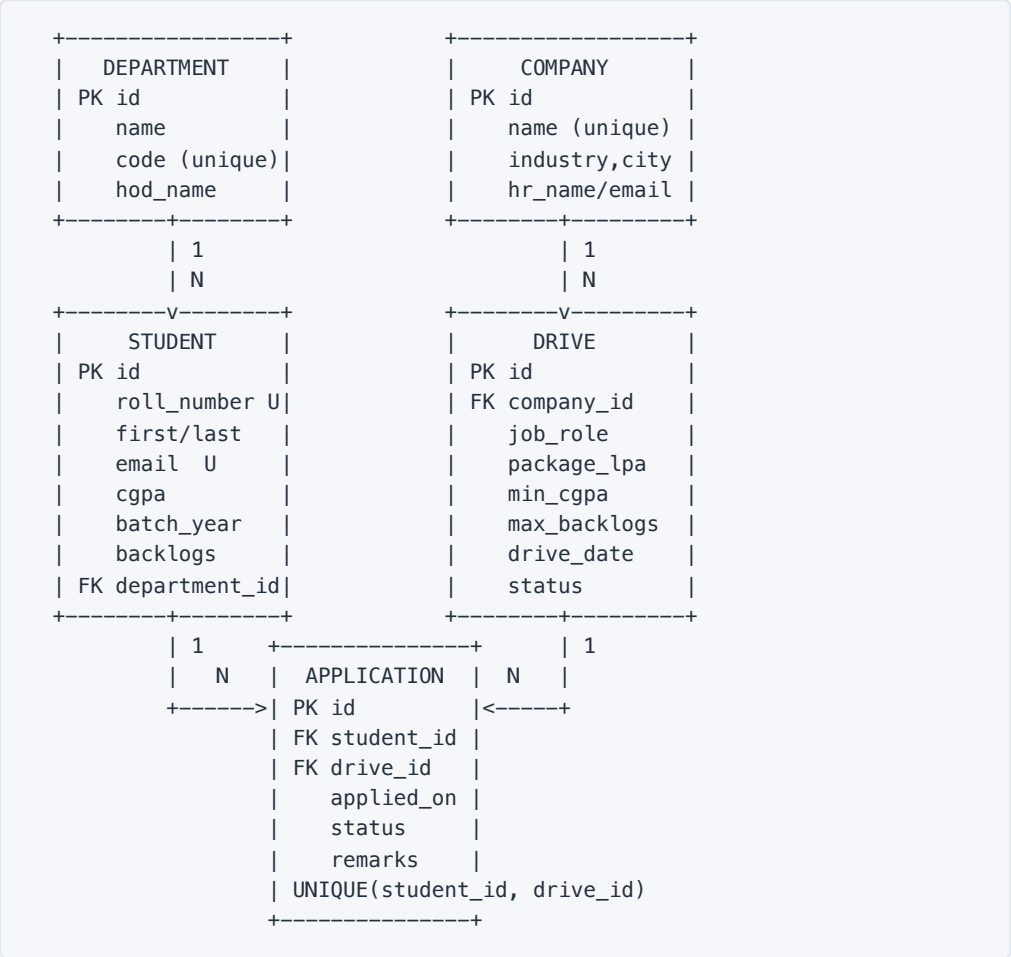




Placement reports



ER design - five tables



RELATIONSHIPS

BETWEEN	TYPE	MEANING
Student → Department	N : 1	Many students in one branch
Drive → Company	N : 1	One company, many drives
Student ↔ Drive	M : N	Resolved by **application**

> **Why a junction entity and not `@ManyToMany`** - the link itself carries data: `applied_on`, `status` and `remarks`. A plain many to many table can only store the two keys.

SAMPLE DATA LOADED

20	8	10	24
STUDENTS	COMPANIES	DRIVES	APPLICATIONS

Normalisation - from one register to five tables

UNNORMALISED REGISTER

```
roll_no, student_name, branch, hod, cgpa,
company, hr_name, hr_email, role, package,
drive_date, status, remark
```

1NF - ATOMIC VALUES, NO REPEATING GROUPS

`student_name` split into `first_name` and `last_name`. A student applying to three companies is not stored as company1, company2, company3 - each application becomes its own row.

2NF - REMOVE PARTIAL DEPENDENCY

With key (roll_no, company): `cgpa` and `branch` depend only on roll_no, while `hr_name` and `hr_email` depend only on company. So the register was split into **student**, **company** and **drive**, each with a single column key. **application** keeps only what needs both keys - applied_on, status, remarks.

3NF - REMOVE TRANSITIVE DEPENDENCY

roll_no → branch → hod. Branch data moved to the **department** table and the student row now stores only `department_id`.

WHAT NORMALISATION GAVE US

ANOMALY	BEFORE	AFTER
Update	HR email of a company repeated in every drive row	One row update in **company**
Insert	A company could not be recorded before it announced a drive	Company row is independent
Delete	Deleting the last application lost the company details	Only the link row is deleted
Redundancy	Branch name and HOD typed for every student	Stored once, referenced by id

KEYS AND CONSTRAINTS

- Primary key - identity `id` on all five tables.
- Candidate keys - `roll_number`, `email`, `company.name`, `department.code`.
- Foreign keys - `student.department_id`, `drive.company_id`, `application.student_id`, `application.drive_id`.
- Composite unique key - `(student_id, drive_id)` in application.
- `ON DELETE` handled by JPA cascade with `orphanRemoval = true`.

Joins used in the project

A) STUDENT WITH BRANCH - INNER JOIN, 2 TABLES

```
SELECT s.roll_number, s.first_name, s.cgpa, d.code
FROM student s
JOIN department d ON d.id = s.department_id
ORDER BY s.roll_number;
```

B) CONFIRMED OFFERS - 3 TABLE JOIN

```
SELECT s.roll_number, c.name AS company,
       dr.job_role, dr.package_lpa
FROM application a
JOIN student s  ON s.id  = a.student_id
JOIN drive   dr ON dr.id = a.drive_id
JOIN company c  ON c.id  = dr.company_id
WHERE a.status = 'SELECTED'
ORDER BY dr.package_lpa DESC;
```

In JPQL the same query uses `join fetch`, which loads all four tables in one select instead of one query per row (the N+1 problem).

C) BRANCH WISE REPORT - LEFT JOIN + GROUP BY

```
SELECT d.code,
       COUNT(DISTINCT s.id) AS total,
       COUNT(DISTINCT CASE WHEN a.status='SELECTED'
                           THEN s.id END) AS placed
FROM department d
LEFT JOIN student s     ON s.department_id = d.id
LEFT JOIN application a ON a.student_id = s.id
GROUP BY d.code ORDER BY d.code;
```

LEFT JOIN is needed so a branch with zero placements (ECE in the sample data) still shows in the report with 0%.

D) ELIGIBLE STUDENTS - JOIN WITH NOT IN SUB QUERY

```
SELECT s.* FROM student s
WHERE s.cgpa >= 7.0 AND s.backlogs <= 0
  AND s.id NOT IN (SELECT a.student_id
                   FROM application a
                   WHERE a.drive_id = 3);
```

Note on DISTINCT - the placed count uses `COUNT(DISTINCT a.student_id)`, because a student selected by two companies must still be counted as one placed student.

Results, testing and future scope

AUTOMATED TESTS - ALL PASSING

TEST	WHAT IT PROVES
studentCrudWorks	Insert → read → update → delete, then 404 for the deleted id
duplicateRollNumberIsRefused	Repeated roll number is rejected
duplicateApplicationIsRejected	Same student cannot apply twice to one drive
lowCgpaApplicationIsRejected	6.2 CGPA student refused for a 7.0 cut off drive
placedCountMatchesSelected	Report count equals the SELECTED rows

```
Tests run: 5, Failures: 0, Errors: 0
```

MANUAL TESTING

Every CRUD path in all four modules, search and filter, drive status switch, applying from the eligible list, empty form submit, duplicate entry, ineligible application and a wrong URL.

WHAT THE SAMPLE SEASON SHOWS

40%
BATCH PLACED

7.0
HIGHEST LPA

4.84
AVERAGE LPA

FUTURE SCOPE

- Spring Security login - officer, coordinator and student see different screens.
- Student self service: apply directly and track the status.
- Resume upload as a file instead of a link.
- Email / SMS alert when a drive opens or a result is declared.
- Export the report tables to Excel or PDF.
- Switch to MySQL for permanent storage and add Flyway for schema versions.

Conclusion - the project covers the full cycle of a placement season with clean CRUD operations, a normalised database, business rules enforced in one place and reports that are generated from joins instead of being counted by hand.